



MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN SUPERIOR

UNIVERSIDAD SIMÓN BOLÍVAR

COORDINACIÓN DE INGENIERÍA DE LA COMPUTACIÓN

SARTENEJAS, DISTRITO CAPITAL.

PROYECTO DE LABORATORIO

Informe Técnico - Corte 2

SecureVision C.A. - Sistema de Vigilancia con IA

Docente: Juan Andrés Cuevas.

Taller de Bases de Datos I (CI-3391)

Estudiantes: Alejandro Villamizar 22-10439

Cristina Puyosa 23-10395 .

Carlos Bisogno 22-100041 .

Sartenejas, junio 2026.

Informe Técnico - Corte 2

Cambios respecto al Corte anterior

A continuación se describen los cambios aplicados al modelo físico en respuesta a las correcciones del profesor y a decisiones propias del equipo durante la implementación SQL.

Renombramiento de atributos al inglés

El profesor indicó como recomendación de mejora que los nombres de atributos debían estar en inglés para mayor legibilidad y consistencia con el código SQL. Se renombraron todos los atributos siguiendo convenciones descriptivas:

Corte 1 (español)	Corte 2 (inglés)
nocturna	has_night_vision
tipoZona	zone_type
eTiempo	eTime
confianzaNiv	conf_level
ancho / alto	width / height
tipoObj	object_type
equipaje	luggage
vehiculo (atrib.)	vehicle
matricula	license_plate
huellaVisual	embedding_vec

Cambio de SERIAL a UUID como clave primaria

En el Corte 1 se usaron claves SERIAL (enteros autoincrementales). En la implementación se migraron todas las PKs a UUID generados con `gen_random_uuid()`. Esta decisión ofrece las siguientes ventajas:

- Evita colisiones al integrar datos de múltiples instancias o entornos de prueba sin necesidad de re-secuenciar.
- Facilita la futura distribución horizontal del sistema, donde secuencias autoincrementales generan conflictos.

- Es más seguro exponer en una API RESTful (un UUID no revela el volumen de registros ni permite enumeración trivial).

Separación del embedding en tabla independiente (Embedding)

El profesor señaló que el atributo `huellaVisual` (embedding de 512 dimensiones) siempre sería NULL cuando el nivel de confianza del evento sea menor al 60 %, lo que convierte ese campo en un atributo frecuentemente vacío. En respuesta, se extrajo el embedding a una tabla separada `embedding`:

```
create table embedding(  
    EmID UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
    OID UUID unique not null,  
    constraint fk_embedding_object foreign key (OID) references "object"(OID),  
    embedding_vec vector(512) not null  
);
```

La restricción UNIQUE sobre OID garantiza que existe como máximo un embedding por objeto, modelando una relación 1:0..1. Así, la tabla `object` no contiene columnas nulas estructuralmente masivas y el embedding simplemente no existe como fila si no se calculó.

Enumeraciones: CHECK vs tipo ENUM nativo

El profesor mencionó como sugerencia el uso de tipos ENUM nativos de PostgreSQL. El equipo optó deliberadamente por mantener restricciones CHECK (`col IN (...)`) por las siguientes razones:

- Los tipos ENUM en PostgreSQL son inmutables sin DDL: agregar un nuevo valor (p.ej. un nuevo tipo de vehículo) requiere ALTER TYPE, que puede bloquear la tabla en producción.
- Las restricciones CHECK permiten evolucionar el dominio de forma más controlada y son más portables entre motores SQL.
- Para los valores de estado y severidad —que son fijos por el enunciado— la diferencia práctica es mínima, y la legibilidad del CHECK es suficiente.

Corrección de atributos de subclases en Objeto

En el corte 1 el profesor indicó que faltaba especificar los posibles valores de `vehicle` (tipo de vehículo) y `luggage` (tipo de equipaje). Se agregaron restricciones CHECK explícitas sobre ambos campos:

```
luggage varchar(20) null check (luggage in ('mochila', 'maleta', 'maletin')),  
"vehicle" varchar(20) null check ("vehicle" in ('motocicleta', 'sedan', 'SUV', 'autobus', 'camion')),
```

Los valores provienen directamente del archivo csv del proyecto y cubren los casos de uso descritos para personas y vehículos respectivamente.

Adición del atributo 'name' en Location y Camera

Consideramos que en el modelo del corte 1 el atributo edificio en ubicación no describía correctamente este campo ya que en el archivo .csv habían zonas cuyos nombres no eran el de un edificio (ejemplo: Pasillo Peatonal Sur), por tanto se optó por cambiar el nombre del atributo 'edificio' por 'name'

Por su parte, se prefirió colocar el nombre de la cámara como un atributo y no como un UUID, para desacoplar el identificador técnico del nombre operativo visible al operador. También porque el atributo nombre en el archivo .csv presenta repetidos. Por ende se coloca en el esquema solo como atributo simple 'name' y no como una clave primaria como se tenía anteriormente. Como consecuencia, se coloca un serial generado aleatoriamente como UUID llamado CID para reemplazar el nombre de la cámara que anteriormente se tenía como UUID.

Cambio del piso a VARCHAR(3)

En el Corte 1, el atributo piso era INTEGER. Se cambió a VARCHAR(3) para aceptar identificadores alfanuméricos comunes en edificios reales (ejemplo: 'S1' para sótano 1, 'PB' para planta baja). Esto es más flexible que un entero, además sigue siendo compacto y cumple con el formato del archivo .csv.

Esquema físico final

A continuación se presentan las tablas del esquema con sus atributos, tipos de dato y restricciones de integridad.

Tabla location

Atributo	Tipo	Restricciones	Descripción
UID	UUID	PK, DEFAULT gen_random_uuid()	Identificador único de la ubicación
name	VARCHAR(100)	NOT NULL	Nombre descriptivo del lugar

Atributo	Tipo	Restricciones	Descripción
floor	VARCHAR(3)	NOT NULL	Piso o nivel (acepta 'PB', 'B1', '3', etc.)
zone_type	VARCHAR(50)	NOT NULL	Categoría de zona (comercial, estacionamiento...)
latitude	DECIMAL(10, 7)	NOT NULL	Latitud geográfica (componente de coordenadas)
longitude	DECIMAL(10, 7)	NOT NULL	Longitud geográfica (componente de coordenadas)

Tabla camera

Atributo	Tipo	Restricciones	Descripción
CID	UUID	PK, DEFAULT gen_random_uuid()	Identificador único de la cámara
UID	UUID	FK → location(UUID)	Ubicación donde está instalada
name	VARCHAR(20)	NOT NULL	Nombre operativo de la cámara
model	VARCHAR(100)	NOT NULL	Modelo del dispositivo físico
has_night_vision	BOOLEAN	NOT NULL, DEFAULT FALSE	Capacidad de visión nocturna
state	VARCHAR(20)	NOT NULL, DEFAULT 'activa', CHECK IN ('activa','en_mantenimiento','inactiva')	Estado operativo

Tabla event

Atributo	Tipo	Restricciones	Descripción
EID	UUID	PK, DEFAULT gen_random_uuid()	Identificador único del evento

Atributo	Tipo	Restricciones	Descripción
CID	UUID	FK → camera(CID)	Cámara que capturó el frame
eTime	TIMESTAMP Z	NOT NULL, DEFAULT current_timestamp	Instante del evento con zona horaria
conf_level	DECIMAL(4,3)	NOT NULL, CHECK BETWEEN 0 AND 1	Nivel de confianza del modelo (0.000–1.000)
posX	INTEGER	NOT NULL	Coordenada X del recuadro (píxeles)
posY	INTEGER	NOT NULL	Coordenada Y del recuadro (píxeles)
width	INTEGER	NOT NULL	Ancho del recuadro (píxeles)
height	INTEGER	NOT NULL	Alto del recuadro (píxeles)

Tabla alert

Atributo	Tipo	Restricciones	Descripción
AID	UUID	PK, DEFAULT gen_random_uuid()	Identificador único de la alerta
EID	UUID	FK → event(EID)	Evento que originó la alerta
aTime	TIMESTAMP Z	NOT NULL, DEFAULT current_timestamp	Instante de generación de la alerta
severity	VARCHAR(10)	NOT NULL, CHECK IN ('baja', 'media', 'alta', 'critica')	Nivel de gravedad
state	VARCHAR(15)	NOT NULL, DEFAULT 'pendiente', CHECK IN ('pendiente', 'atendida', 'des cartada')	Estado de atención
description	TEXT	NOT NULL	Descripción textual del motivo

Tabla object

Atributo	Tipo	Restricciones	Descripción
OID	UUID	PK, DEFAULT gen_random_uuid()	Identificador único del objeto
EID	UUID	FK → event(EID), NOT NULL	Evento al que pertenece
object_type	VARCHAR(10)	NOT NULL, CHECK IN ('person','vehicle')	Discriminador de subclase
color	VARCHAR(50)	NULL	Color de vestimenta (persona) o carrocería (vehículo)
luggage	VARCHAR(20)	NULL, CHECK IN ('mochila','maleta','malet in')	Tipo de equipaje. Solo si object_type = 'person'
vehicle	VARCHAR(20)	NULL, CHECK IN ('motocicleta','sedan','SU V','autobus','camion')	Tipo de vehículo. Solo si object_type = 'vehicle'
license_plate	VARCHAR(20)	NULL	Matrícula leída. Solo si object_type = 'vehicle'

Tabla embedding

Atributo	Tipo	Restricciones	Descripción
EmID	UUID	PK, DEFAULT gen_random_uuid()	Identificador único del embedding
OID	UUID	FK → object(OID), UNIQUE NOT NULL	Objeto al que corresponde (1:0..1)
embedding_vec	VECTOR(512)	NOT NULL	Vector de 512 dimensiones para búsqueda por similitud coseno

Otras decisiones de diseño

Jerarquía de Objeto: tabla única con discriminador

Al igual que en el corte 1 la especialización total y disjunta de Objeto en Persona y Vehículo se implementó en una única tabla con el campo `object_type` como discriminador (patrón Single Table Inheritance). La alternativa de tablas separadas `PERSON` y `VEHICLE` con `JOIN` fue descartada porque:

- La cantidad de atributos diferenciales es reducida (2 en persona, 3 en vehículo), generando pocas columnas `NULL` por fila.
- Las consultas analíticas más frecuentes ("¿cuántos objetos detectó esta cámara?") no necesitan distinguir el subtipo; el `JOIN` adicional sería costoso y sin beneficio.
- El almacenamiento extra de unas columnas `NULL` es despreciable frente al ahorro en tiempo de CPU de los `JOINS`.

El campo compartido `color` no genera ambigüedad porque la especialización es disjunta: un registro nunca es persona y vehículo simultáneamente, por lo que el significado semántico de `color` queda determinado por `object_type`.

Restricción compuesta `chk_object_attributes`

Para garantizar la integridad semántica de la jerarquía se define la siguiente restricción de tabla:

```
constraint chk_object_attributes
check ((object_type = 'person'
        and vehicle is null
        and license_plate is null)
       or
       (object_type = 'vehicle'
        and luggage is null))
);
```

Esta restricción implementa dos reglas de negocio derivadas del enunciado:

- Si el objeto es una persona, los campos exclusivos de vehículo (`vehicle`, `license_plate`) deben ser `NULL`.
- Si el objeto es un vehículo, el campo exclusivo de persona (`luggage`) debe ser `NULL`.

Nótese que `color` se permite en ambos subtipos (color de vestimenta o color de carrocería), por lo que no aparece en la restricción. Esta es la razón por la que `color` es un atributo compartido en lugar de duplicarse en dos columnas.

TIMESTAMPTZ en lugar de TIMESTAMP

Todos los campos temporales (eTime, aTime) usan TIMESTAMPTZ (timestamp con zona horaria). Siguiendo la recomendación técnica del enunciado, esto garantiza consistencia cuando las instalaciones de SecureVision operan en zonas horarias diferentes. PostgreSQL almacena internamente en UTC y convierte a la zona horaria de la sesión en la lectura, lo que evita ambigüedades en registros de auditoría y análisis forense de alertas.

DECIMAL(4,3) para conf_level

El nivel de confianza es un valor continuo entre 0 y 1 con tres decimales de precisión (e.g. 0.983). Se usa DECIMAL(4,3): 4 dígitos significativos en total, 3 después del punto decimal. Esto cubre el rango completo [0.000, 1.000] sin desperdicio de almacenamiento. La restricción CHECK (conf_level BETWEEN 0 AND 1) impide valores fuera de rango que podrían resultar de errores en el pipeline de ingesta.

Índices creados

Se crearon cuatro índices para optimizar las consultas analíticas más frecuentes previstas:

```
CREATE INDEX ON "event"(CID);      -- Consultas por cámara
CREATE INDEX ON "event"(eTime);    -- Consultas temporales y rangos de tiempo
CREATE INDEX ON "location"(zone_type); -- Filtros por tipo de zona
CREATE INDEX ON embedding
  USING hnsw (embedding_vec vector_cosine_ops); -- Búsqueda ANN por similitud
coseno
```

El índice HNSW (Hierarchical Navigable Small World) sobre embedding_vec es el más relevante arquitecturalmente: permite búsquedas de vecinos más cercanos aproximados (ANN) con complejidad sub-lineal en lugar de un scan secuencial, lo que es crítico para consultas del tipo "encuentra objetos visualmente similares a este". Se usa distancia coseno (vector_cosine_ops) porque los embeddings de modelos de visión son vectores de dirección donde la similaridad angular es más significativa que la distancia euclídea.

Extensión pgvector

El esquema requiere la extensión pgvector de PostgreSQL para el tipo VECTOR(512). Por ello, el script inicia con:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

El uso de IF NOT EXISTS hace el script idempotente: puede ejecutarse en entornos donde la extensión ya esté instalada sin producir errores.

DROP TABLE ... CASCADE al inicio del script

El script de esquema comienza con sentencias DROP TABLE para facilitar el ciclo de desarrollo:

```
drop table if exists "location" cascade;  
drop table if exists camera cascade;  
drop table if exists "event" cascade;  
drop table if exists alert cascade;  
drop table if exists "object" cascade;  
drop table if exists embedding cascade;
```

La cláusula CASCADE elimina automáticamente las restricciones de clave foránea que dependan de la tabla eliminada. Esto es necesario porque el orden de eliminación no siempre es el inverso del orden de creación. Estas sentencias no deben ejecutarse en producción; están pensadas exclusivamente para el entorno de desarrollo y pruebas.

Relaciones e integridad referencial

Relación	Cardinalidad	Implementación	Obligatoriedad
location → camera	(1,1) : (0,N)	camera.UID → location.UID	Una cámara siempre tiene ubicación (FK NOT NULL implícita por diseño)
camera → event	(1,1) : (0,N)	event.CID → camera.CID	Un evento siempre proviene de una cámara
event → object	(1,1) : (1,N)	object.EID → event.EID	Todo objeto pertenece a un evento; un evento tiene al menos un objeto
event → alert	(1,1) : (0,N)	alert.EID → event.EID	Una alerta siempre referencia un evento; un evento puede no generar alertas
object → embedding	(1,1) : (0,1)	embedding.OID → object.OID UNIQUE	Un objeto puede tener a lo sumo un embedding (solo si conf_level ≥ 60%)